



PEOPLE'S DEMOCRATIC REPUBLIC OF ALGERIA

**MINISTRY OF HIGHER EDUCATION AND
SCIENTIFIC RESEARCH**

UNIVERSITY ZIANE ACHOUR – DJELFA

**FACULTY OF EXACT SCIENCES AND COMPUTER
SCIENCE DEPARTMENT**

SPECIALTY: SYSTEMS OF INFORMATION

Third year **LICENSE ACADEMIC's** Report

"TimeSplitter-Beta"

**A Beta Version for an Application for Automatic University Timetable
Generation**

Presented by:

HARFOUCHE AHMED

YEAR: 2025/2026

Acknowledgement

I would like to extend my sincere gratitude to all the professors and academic staff of the Faculty of Exact Sciences and Computer Science at the University of Ziane Achour – Djelfa, whose guidance and teachings throughout my academic journey have been invaluable.

I am deeply thankful to my colleagues for their encouragement and constructive discussions that helped shape this project. Special thanks go to my family for their continuous support and patience throughout this period.

This project is the result of personal dedication, and a genuine desire to contribute a practical solution to a real academic problem. Above all, I thank god for granting me the strength and the knowledge that make me complete this work.

Dedication

To my family, who stood by me through every challenge and never stopped believing in me.

To myself — for choosing to build something real, something useful, instead of taking the easy road. For the late nights, the debugging sessions, and the moments of doubt that I pushed through. You made it.

Table of Contents

Acknowledgement	1
Dedication	2
Table of Contents	3
General Introduction	4
Chapter 1: Research Context.....	5
1.1 Introduction.....	5
1.2 Problem Statement	5
1.3 Existing Solutions	5
1.4 Proposed Solution: TimeSplitter-Beta	6
1.5 Conclusion	7
Chapter 2: Analysis and Design.....	8
2.1 Introduction.....	8
2.2 Functional Requirements	8
2.3 Non-Functional Requirements	8
2.4 Actors.....	9
2.5 Use Case Diagram.....	9
2.6 Class Diagram.....	10
2.7 Sequence Diagram	11
2.8 Conclusion	12
Chapter 3: Implementation	13
3.1 Introduction.....	13
3.2 Tools and Technologies	13
3.3 Application Architecture.....	13
3.4 Key Algorithms.....	15
3.4.1 CM Session Assignment	15
3.4.2 TD/TP Assignment: Round Robin Algorithm	16
3.4.3 PDF Export	18
3.5 Application Presentation.....	18
3.6 Conclusion	19
General Conclusion.....	20
List of Figures.....	21
Bibliography	22

General Introduction

In every university, managing the academic timetable is one of the most complex and time-consuming administrative tasks. Professors, department heads, and administrative staff often spend days or even weeks manually constructing schedules that avoid conflicts, respect constraints, and satisfy the needs of all student groups.

At the University of Ziane Achour – Djelfa, like most Algerian universities, the academic system is organized around modules (courses), student groups (Groupes), and three types of sessions: CM (Cours Magistral – lecture), TD (Travaux Dirigés – directed work)/ TP (Travaux Pratiques – practical work). Each type follows specific scheduling rules, and coordinating all of them manually is a significant challenge.

This project, TimeSplitter-Beta, proposes a beta version of an automated application that intelligently generates university class schedules. All the user needs to do is enter a list of courses, specify which courses include practical sessions, and define the number of student groups. The application then automatically generates a complete, conflict-free weekly schedule using a Round Robin scheduling algorithm.

Chapter 1: Research Context

1.1 Introduction

Organizing university class schedules is a crucial task, as it directly impacts the quality of education. A well-structured schedule ensures that students attend all their lectures without scheduling conflicts, avoids overlapping professor schedules, and optimizes classroom use.

In this chapter, we explore the challenges of university class scheduling, examine current approaches, and present TimeSplitter-Beta as a proposed partial or experimental solution.

1.2 Problem Statement

Preparing university course schedules is a well-known and complex problem in computer science. This problem falls under the category of constraint-satisfaction problems and is classified as NP-Hard, meaning that finding an optimal solution becomes significantly more difficult as the number of constraints increases.

In the Algerian university system, this challenge is particularly complex due to the following factors:

Multiple Session Types: Each course module may contain up to three types of sessions: a lecture (CM) for all groups together, a tutorial (TD) for one group in each room, and a practical (TP) for one group in each room. Each type follows different scheduling rules.

Multiple Student Groups: A single cohort may be divided into several groups (called "cohorts" or "groups"), typically ranging from one to five students. Each group must attend its own TD/TP sessions, meaning that the same course module may require booking multiple rooms at different times.

Fair distribution: Each student group must attend only one practical/theoretical session per course. No group may attend two sessions for the same course, and no two groups may study the same course simultaneously.

Currently, some universities manage this process manually, making it slow, error-prone, and difficult to update when changes occur.

1.3 Existing Solutions

Several software tools exist for timetable generation, including:

- **FET (Free Timetabling Software):** A powerful open-source tool, but complex to configure .
- **UniTime:** A comprehensive academic scheduling system used by large universities, but requires significant technical infrastructure.
- **Manual Excel sheets:** The most common approach in Algerian universities, flexible but extremely time-consuming and prone to human error.

None of these solutions offers a simple, targeted tool adapted to the specific structure of Algerian universities.

1.4 Proposed Solution: TimeSplitter-Beta

TimeSplitter-Beta is a Beta-Version for desktop application developed in C# using Windows Forms. It is designed specifically for the Algerian university context and this version offers the following capabilities:

- **Semi-Modern_UI input interface:** The user enters the number of modules, the name of each module, whether it has a practical session (TD/TP), and the number of student groups.
- **Automatic timetable generation:** The application automatically assigns CM sessions to morning slots and TD/TP sessions to afternoon slots, distributing them across the week.
- **Round Robin scheduling:** A fair rotation algorithm ensures that each student group attends every module's TD/TP session exactly once, with no conflicts.
- **Visual timetable display:** The generated timetable is displayed in a clear, readable grid format (days × time slots).
- **Exporting:** The user can save the generated timetable as a PDF file for printing or sharing.
- **Languages support :** The application supports both English and Arabic .

1.4.1 Application Objectives

1. Eliminate scheduling conflicts between student groups.
2. Ensure fair distribution of TD/TP sessions across all groups using the Round Robin algorithm.
3. Provide a simple, intuitive interface usable by non-technical staff.
4. Allow export of the generated timetable to PDF format.

1.4.2 Problems and Solutions

Problem	Solution in “TimeSplitter-Beta”
Manual timetable creation is slow and error-prone	Automatic generation with one click
TD/TP conflicts between groups	Round Robin algorithm guarantees no conflicts
Difficult to share or print timetables	Built-in PDF export feature
Language barrier for Arabic-speaking staff	Full Arabic interface option

1.5 Conclusion

The problem of university timetable generation is a genuine and recurring challenge in Algerian academic institutions. “TimeSplitter-Beta” addresses this problem with a simple solution. By combining an intuitive interface with a mathematically sound scheduling algorithm, it significantly reduces the time and effort required to produce a valid weekly timetable.

Chapter 2: Analysis and Design

2.1 Introduction

In this chapter, we present the analysis and design of “TimeSplitter-Beta”. We define the functional and non-functional requirements, identify the system actors, and illustrate the system behavior using UML diagrams including Use Case, Class, and Sequence diagrams.

2.2 Functional Requirements

The system must allow the user to:

- Enter the number of modules (maximum 8) and the number of student groups (maximum 6).
- Enter the name of each module and specify whether it has a TD/TP (practical session).
- Generate a complete weekly timetable automatically with a single click.
- View the generated timetable in a clear grid format (days as rows, time slots as columns).
- Export the timetable to a PDF file and save it to a chosen location.
- Switch the language between English and Arabic.
- Cancel and reset all inputs at any time.

2.3 Non-Functional Requirements

- **Usability:** The interface must be simple enough for non-technical administrative staff.
- **Performance:** Timetable generation must complete instantly (under 1 second).
- **Reliability:** The algorithm must always produce a conflict-free timetable.
- **Portability:** The application runs on Windows operating systems.
- **Maintainability:** The code is structured with separate forms and reusable components.

2.4 Actors

Administrator / Department Head: The primary user of the application. They input module information, configure student groups, generate the timetable, and export the result.

2.5 Use Case Diagram

The main use cases of “TimeSplitter-Beta” are:

- Manage Modules Data
- Enter Modules numbers
- Enter modules names
- Set TD/TP Status
- Generate timetable
- Configure Timetable Settings
- Determining Groupes Numbers
- Table Cells Color
- Export to PDF
- Switch Language

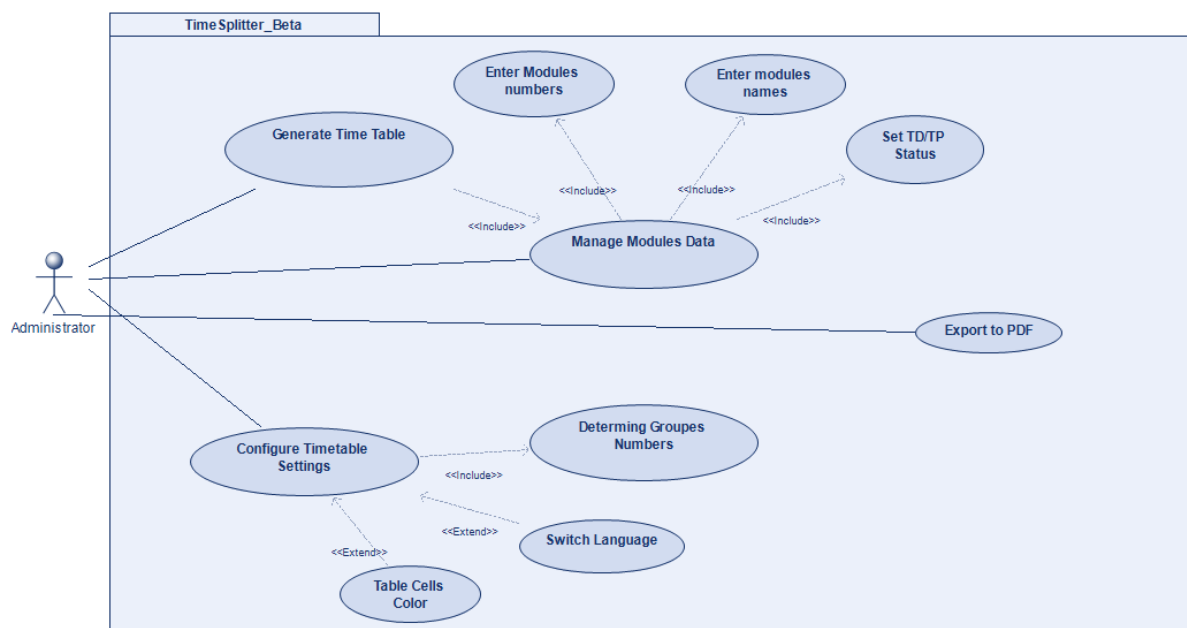


Figure 1

Use Case Description: Generate Timetable:

Field	Description
Actor	Administrator / Department Head
Objective	Generate an automatic weekly timetable
Pre-condition	Number of modules and groups entered; module names provided

Field	Description
Post-condition	A complete timetable is displayed in the result window
Nominal Scenario	1. User enters module count and group count. 2. User clicks OK. 3. Input fields appear. 4. User enters module names and checks TD/TP boxes. 5. User clicks "Make Calendar". 6. System generates timetable and opens result window.
Alternative Scenario	User clicks Cancel to reset all inputs
Error Scenario	No modules entered (if this happened); system produces empty timetable , (the system will tell you to enter modules)

Use Case Description: Export to PDF

Field	Description
Actor	Administrator / Department Head
Objective	Save the generated timetable as a PDF file
Pre-condition	Timetable has been generated and is displayed
Post-condition	A PDF file is saved at the chosen location
Nominal Scenario	1. User clicks "Save as" button. 2. System opens file dialog. 3. User chooses location and file name. 4. System captures the timetable and saves it as PDF. 5. Success message pop out.
Alternative Scenario	User cancels the file dialog; no file is saved

2.6 Class Diagram

The main data structures used in TimeSplitter-Beta are:

ModulesData (struct)

- moduleName : string
- TDTPIsChecked : bool
- dayNum : int
- hourRank : int

Form1 (Main Form)

- ModData : List<ModulesData>
- NumOfGrp : int
- btnMake_Click()
- Ok_Click()

ResultForm (Result Window)

- GenerateTable()
- FillTable(ModData, numOfGrp, ...)
- SaveToPDF()

labelUserControl (Custom Control)

- ModuleName : string
- IsTDTPChecked : bool

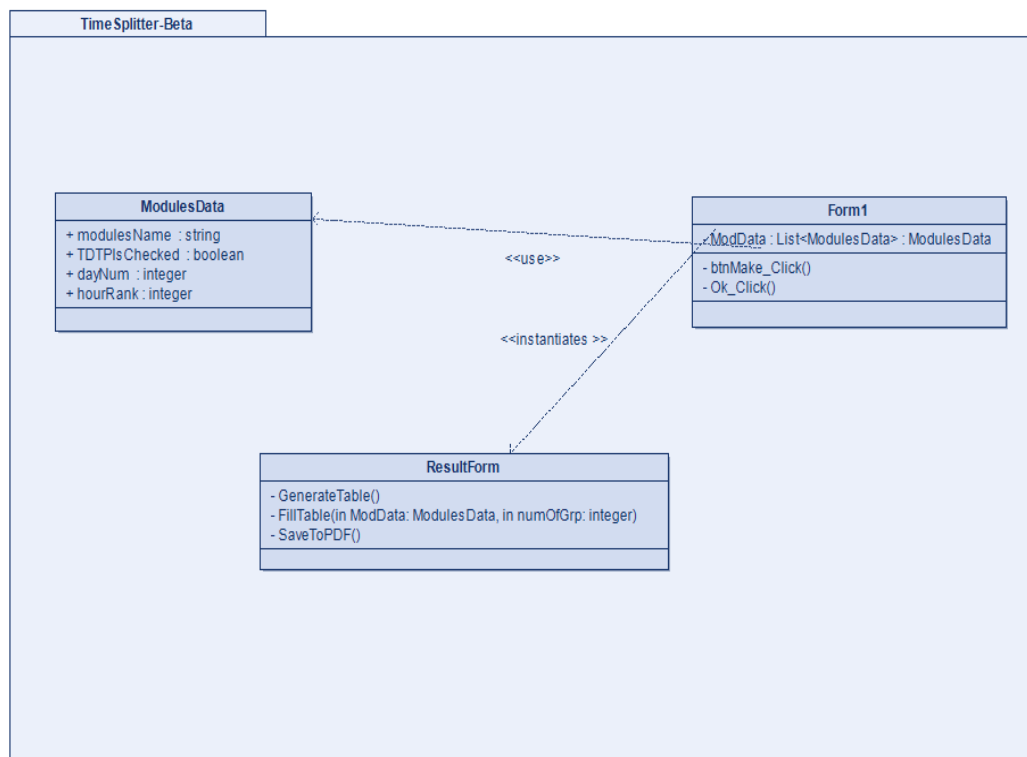


Figure 2

2.7 Sequence Diagram

Sequence: Generate and Display Timetable

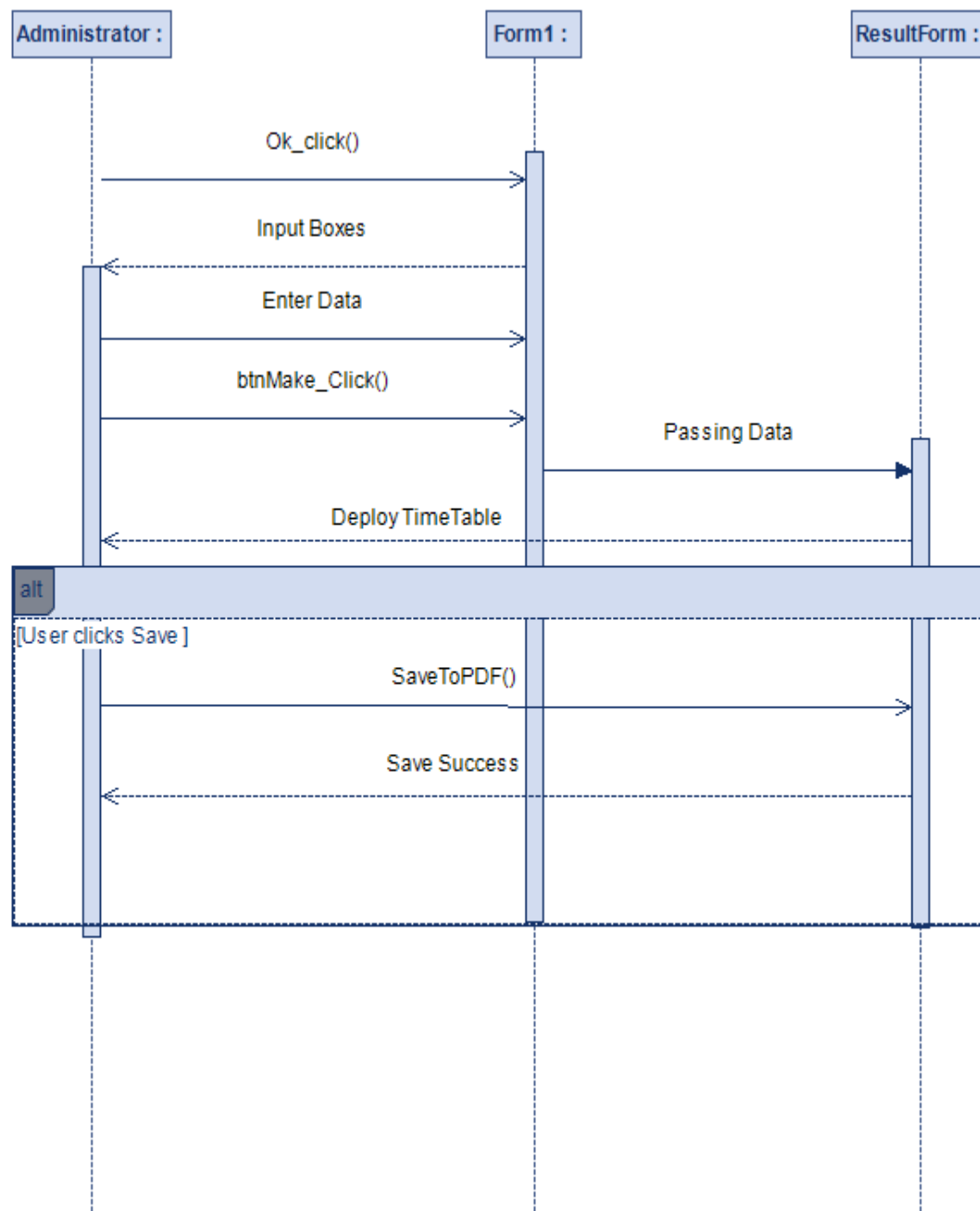


Figure 3

2.8 Conclusion

The analysis phase confirmed that TimeSplitter-Beta requires a simple but carefully designed data model and a mathematically sound scheduling algorithm. The use of structs for module data, separate forms for input and output, and the Round Robin algorithm for TD/TP distribution form the the system's design.

Chapter 3: Implementation

3.1 Introduction

This chapter presents the technical implementation of TimeSplitter-Beta. We describe the tools and technologies used, explain the key algorithms, and present the application interface through screenshots.

3.2 Tools and Technologies

Tool	Description	Usage
C#	Object-oriented programming language by Microsoft	Main application logic
Windows Forms (WinForms)	UI framework for desktop applications on Windows	Building the graphical interface
Visual Studio 2022	Integrated Development Environment	Writing, debugging, and building the application
iTextSharp	.NET library for PDF generation	Exporting the timetable to PDF
DataGridView	Built-in WinForms control	Displaying the timetable grid
NuGet Package Manager	Package manager for .NET	Installing iTextSharp

3.3 Application Architecture

TimeSplitter-Beta is structured as a two-form Windows Forms application:

Form1 (Main Input Form):

TimeSplitter_Beta

English

Group Count : 4

Ok

Coloring the table cells

Module TD/TP

A ☒

B ☒

C ☒

D ☐

E ☐

Cancel

User guide:

If you don't know how to use this application you just have to enter the number of modules in the box on top-left, and click Ok , will appear an input bars and two box next to every bar . and then you have to enter every module's name and check the boxes ,if the module has a TD/TP as you see above the check boxes, and you can check both when you finish click "Make Calendar" green button and will get your result

Make Calendar

Version: Beta

Exit

Figure 4

Allows the user to configure modules and groups

Contains (main content): module count spinner, group count spinner, OK button, dynamic list of module inputs, Make Calendar button, language toggle, and exit button

ResultForm (Timetable Display Form):

Days	8:00-9:30	9:30-11:00	11:00-12:30	Break	13:30-15:00	15:00-16:30
Saturday						
Sunday						
Monday						
Tuesday	A	B	C		G1 TD: A G2 TD: B G3 TD: C	G4 TD: A
Wednesday	D	E			G1 TD: B G2 TD: C	G3 TD: A G4 TD: B
Thursday					G1 TD: C	G2 TD: A G3 TD: B G4 TD: C

Clear Selection

Refresh

Save

Close

Figure 5

Displays the generated timetable in a DataGridView

Contains: the timetable grid , a Save as PDF button, a Refresh button to reBuild a table, and close button to close the Form(ResultForm)

labelUserControl (Custom Control):



Figure 6

A reusable input component for each module

Contains: a text field for the module name and a checkbox for TD/TP

Data Flow

User Input → Form1 → List<ModulesData> → ResultForm → DataGridView → PDF File

3.4 Key Algorithms

3.4.1 CM Session Assignment

CM sessions are assigned to morning time slots using a sequential distribution strategy.

Modules are distributed across the six working days (Saturday to Thursday), with up to 3 CM sessions per day.

```
counter = 0

temp = random starting day(0-5)

For each module:
    if counter >= 3:
        counter = 0
        temp = next day(circular)

    module.dayNum = temp
    module.hourRank = counter + 1
    counter++
```

Figure 7

This ensures that CM sessions are spread across different days and never overlap within the same day-same hour.

3.4.2 TD/TP Assignment: Round Robin Algorithm

The Round Robin algorithm is used to assign TD/TP sessions fairly to all student groups. It guarantees that:

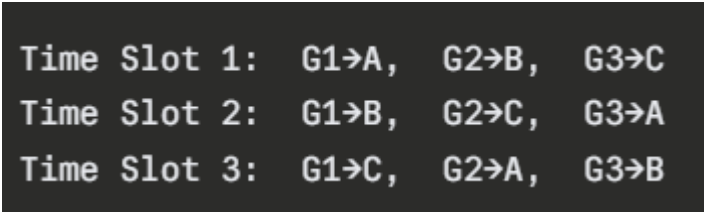
- No two groups study the same module at the same time
- Every group attends every module's TD/TP session exactly once
- Sessions are distributed in afternoon slots only

Mathematical formula:

For time slot i and group g :

$\text{index} = (i + g - 1) \% \text{numberOfTDModules}$

Example with 3 modules (A, B, C) and 3 groups:



```
Time Slot 1:  G1→A,  G2→B,  G3→C
Time Slot 2:  G1→B,  G2→C,  G3→A
Time Slot 3:  G1→C,  G2→A,  G3→B
```

Figure 8

Each group attends each module exactly once, with no conflicts.

Implementation:

```

List<ModulesData> TDTPModules = new List<ModulesData>();
foreach (var m in ModData)
{
    if (m.TDTPIsChecked)
    {
        TDTPModules.Add(m);
    }
}

bool isTDNumGreater;

if (TDTPModules.Count >= numOfGrp)
{
    isTDNumGreater = true;
}
else { isTDNumGreater = false; }

//
if (isTDNumGreater)
{
    int columns = 5;
    int day = ModData[0].dayNum;

    for (int i = 0; i < TDTPModules.Count; i++)
    {
        for (int g = 1; g <= numOfGrp; g++)
        {
            int index = (i + g - 1) % TDTPModules.Count;
            dataGridView1.Rows[day].Cells[columns].Value += "G" + g + " TD/TP: " + TDTPModules[index].moduleName + "\n";
            if (isChecked) dataGridView1.Rows[day].Cells[columns].Style.BackColor = Color.LightGreen;
        }

        if (columns == 5) columns = 6;
        else { columns = 5; day++; }
        if (day > 5) day = 0;
    }
}

```

Figure 9

The Condition if (TDTPModules.Count >= numOFGrp) makes sur that the number of groups is less than number of the TD/TP Modules, because if not the program will conflict the sessions.

Now the (else):

```

else
{
    int columns = 5;
    int day = ModData[0].dayNum;

    for (int i = 0; i < TDTPModules.Count; i++)
    {
        for (int g = 1; g <= numOfGrp; g++)
        {
            int index = (i + g - 1) % TDTPModules.Count;
            dataGridView1.Rows[day].Cells[columns].Value += "G" + g + " TD: " + TDTPModules[index].moduleName + "\n";
            if (isColorBoxChecked) dataGridView1.Rows[day].Cells[columns].Style.BackColor = Color.Lime;

            if (index == TDTPModules.Count - 1)
            {
                if (columns == 5) columns = 6;
                else { columns = 5; day++; }
                if (day > 5) day = 0;
            }
        }

        if (columns == 5) columns = 6;
        else { columns = 5; day++; }
        if (day > 5) day = 0;
    }
}

```

Figure 10

This algorithm handle the state if (TDTPModules.Count < numOFGrp).

3.4.3 PDF Export

```
SaveFileDialog sfd = new SaveFileDialog();

sfd.Filter = "PDF|*.pdf";
sfd.FileName = "Table";
if (sfd.ShowDialog() != DialogResult.OK) { return; }

Bitmap bitmap = new Bitmap(dataGridView1.Width, dataGridView1.Height);
dataGridView1.DrawToBitmap(bitmap, new System.Drawing.Rectangle(0, 0, dataGridView1.Width, dataGridView1.Height));
bitmap.SetResolution(300, 300);

Document doc = new Document(PageSize.A4.Rotate());
PdfWriter.GetInstance(doc, new FileStream(sfd.FileName, FileMode.Create));
doc.Open();

iTextSharp.text.Image img = iTextSharp.text.Image.GetInstance(bitmap, System.Drawing.Imaging.ImageFormat.Png);
img.ScaleToFit(doc.PageSize.Width - 10, doc.PageSize.Height - 10);

img.Alignment = iTextSharp.text.Image.ALIGN_CENTER;
doc.Add(img);

doc.Close();
```

Figure 11

The PDF export feature uses the iTextSharp library. The timetable is captured as a bitmap image from the DataGridView and embedded into an A4 landscape PDF document.

3.5 Application Presentation

Main Interface

The main window of TimeSplitter-Beta allows the user to:

- Set the number of modules using a spinner (top left)
- Set the number of student groups using a second spinner
- Click OK to generate the module input list
- Enter module names and check the TD/TP checkbox for applicable modules
- Toggle between English and Arabic using the language button (blue button)
- Click "Make Calendar" (green button) to generate the timetable
- Exit the application using the Exit button (red button)

A built-in User Guide is displayed in the center of the screen to assist first-time users.

Timetable Result Window

The result window displays the generated timetable as a grid:

- Rows: Days of the week (Saturday to Thursday)
- Columns: Time slots (8:00–9:30, 9:30–11:00, 11:00–12:30, Break, 13:30–15:00, 15:00–16:30)
- Morning slots: CM sessions (module name only)
- Afternoon slots: TD/TP sessions (group number + module name, one per line)

The "Save" button at the bottom right exports the timetable to PDF.

3.6 Conclusion

TimeSplitter-Beta was implemented using C# and Windows Forms, leveraging the built-in DataGridView control for timetable display and the iTextSharp library for PDF export. The core innovation of the application lies in the Round Robin scheduling algorithm, which guarantees a fair, conflict-free distribution of TD/TP sessions across all student groups. The application provides an idea and simple practical tool that suggests a real need in Algerian academic institutions.

General Conclusion

This project aims to propose an idea to solve a tangible and recurring problem in Algerian universities: the manual and error-prone process of creating weekly class schedules. By developing TimeSplitter-Beta, we demonstrated that the targeted software solution can significantly simplify this task.

In the first chapter, we analysed the context of the problem and identified the main challenges in generating schedules, particularly the fair distribution of TD/TP sessions across multiple student groups. In the second chapter, we designed the system using UML diagrams and identified the functional and non-functional requirements. In the third chapter, we implemented the solution using C# and Windows Forms, with the Round Robin algorithm at its core.

The result is a Beta Version of a future functional desktop application that automatically generates a complete weekly schedule in less than one second, without any schedule conflicts.

As future improvements, we envision (if there is no technical problems - or any problems):

- Adding a student group division tool that splits student lists into groups automatically
- Supporting multiple academic levels (L1, L2, L3) simultaneously
- Adding room assignment to avoid room conflicts
- Developing a web-based version for broader accessibility

TimeSplitter-Beta is a small but meaningful step toward the digitalization of academic administration in Algerian universities.

List of Figures

• Figure 1. Use Case Diagram.	9
• Figure 2. Class Diagram.	11
• Figure 3. Sequence Diagram.	12
• Figure 4. TimeSplitter-Beta Main Interface.	14
• Figure 5. Result From interface.	14
• Figure 6. labelUserControl (Custom Control)	15
• Figure 7. CM Session Assignment Code.	15
• Figure 8. Round Robin Distribution Example.	16
• Figure 9. Round Robin Logic (Case: Modules $\geq$ Groups)	17
• Figure 10. Round Robin Logic (Case: Modules $<$ Groups)	17
• Figure 11. PDF export.	18

Bibliography

- [1] Microsoft. (2024). *C# Programming Guide*. Microsoft Docs.
<https://docs.microsoft.com/en-us/dotnet/csharp/>
- [2] Microsoft. (2024). *Windows Forms Documentation*. Microsoft Docs.
<https://docs.microsoft.com/en-us/dotnet/desktop/winforms/>
- [3] iTextSharp. (2023). *iText 5 .NET Documentation*. <https://itextpdf.com/>
- [4] Modelio Open Source . *Modelio Open Source 5.4 – UML Diagrams*
<https://www.modelio.org/>
- [5] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- [6] Wikipedia. (2024). *Round-robin_scheduling*. https://en.wikipedia.org/wiki/Round-robin_scheduling
- [7] Microsoft. (2024). *DataGridView Control*. Microsoft Docs.
<https://docs.microsoft.com/en-us/dotnet/desktop/winforms/controls/datagridview-control>